

CMSC828G: Special Topics in Systems for Machine Learning

Optimizing GPU Kernels

Instructor: Yafan Huang



DEPARTMENT OF
COMPUTER SCIENCE

Recall: What Limits a GPU Kernel?

Computation-Bound

(GEMM, etc)

$$T \approx \frac{\text{Operations}}{\text{Compute Throughput}}$$

Memory-Bound

(Reduction, Compression, etc)

$$T \approx \frac{\text{Bytes Moved}}{\text{Memory Bandwidth}}$$

- Different bottlenecks require different optimizations.

Two Ways to Make a Kernel Faster

Do Less Work

Do the Work Faster

- Compute-bound: Reduce computation Tensor Cores, better utilization
- Memory-bound: Move fewer bytes Coalescing, vectorization, better locality

Compute-Bound: Get More Useful Math per Unit Time

- **Reduce computation**
 - Eliminate redundant operations (or reduced precision).
 - Exploit sparsity / structure when possible.
- **Use faster compute paths**
 - Tensor Cores (or other specialized hardware)
 - Specialized instructions (or other computation primitives)
- **Keep compute units busy**
 - Enough parallelism
 - Instruction-level parallelism
 - Appropriate tiling / workload mapping

Memory-Bound: Move Less Data and Move It Better

- **Move less data**

- Kernel fusion.
- Compression / reduced precision.
- Sometimes recompute instead of reload.

HBM traffic is often the real cost!

- **Move data efficiently**

- Coalesced accesses (or other good data layout).
- Vectorized load / store.

- **Reuse data on-chip**

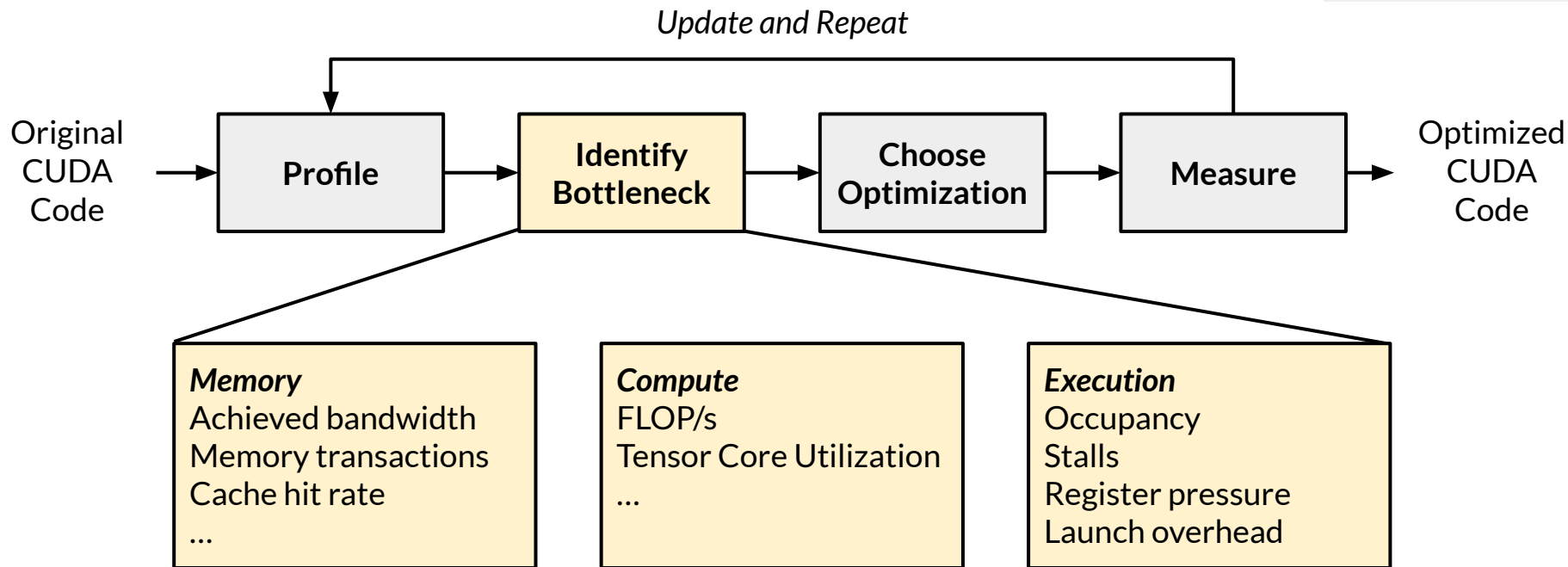
- HBM -> cache -> shared memory -> registers.

There is No Free Optimization

- Kernel Fusion
 - HBM traffic  <-> register pressure 
- Larger Tiles
 - Data reuse  <-> shared memory / register 
- More Parallelism
 - Hardware utilization  <-> possible synchronization overhead 
- Lower Precision
 - Bytes  + compute throughput  <-> accuracy  / conversion overhead 

Optimization is an Iterative Process

*Nsight Compute
Roofline
Triton Autotune*



Never optimize without measuring!

Agentic GPU Kernel Optimization

- LLM generate / propose changes -> profile and measure -> LLM revise -> ...
- Limitations:
 - Expensive search - excessive long context, huge token consumptions
 - No common standard abstraction level
 - Unclear remaining optimization headroom
 - Hardware / shape specific
 - Overly defensive optimization
 - Wrong optimization direction - human expertise / feedback is important!

Student Presentations

- [ICS'24] Accelerated Auto-Tuning of GPU Kernels for Tensor Computations
- [SC'24] cuSZp2: A GPU Lossy Compressor with Extreme Throughput and Optimized Compression Ratio